

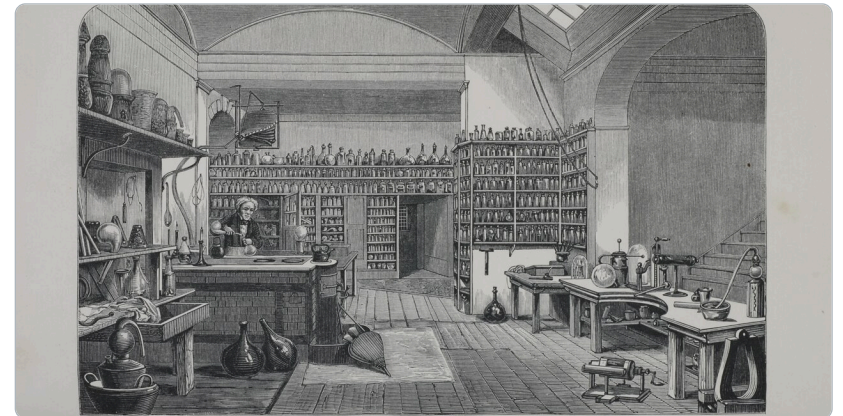
Rejouer une expérience, 6 mois plus tard

Les compromis de la reproductibilité

Jey Puget Gil

Docteur en informatique

› Laboratoire d'Informatique de Grenoble



Gravure, laboratoire de Faraday, 1870

1



PARTIE 1 SUR 4

De quoi parle-t-on ?

Sans aucun prérequis technique



D'où je vous parle

Qui je suis

- Docteure en informatique depuis juin 2026
- Thèse au LIRIS, sur des graphes de connaissances évolutifs
- Pas spécialiste de la reproductibilité

Au programme

- 1 De quoi parle-t-on ? (7 min)
- 2 Mon terrain (4 min)
- 3 Ce que j'ai mis en place (9 min)
- 4 Ce que ça coûte, ce que ça rapporte (5 min)



Héra, protectrice de l'Argo :
d'où **Argo Workflows** et **Hera**.

Une expérience, c'est une recette

Laboratoire d'Informatique de Grenoble ■



Gravure de N. de Larmessin, v. 1695

Un article, c'est la **photo du plat**.

Pour le refaire, il faut ces quatre ingrédients. On n'en publie qu'un ou deux.

⚠ Le point aveugle

La cuisine (système, bibliothèques, versions) n'est presque jamais décrite. Et ses briques bougent.

Les données

Les ingrédients, et leur version exacte

Le code

Les ustensiles et les gestes

L'environnement

La cuisine : système, bibliothèques, versions

Le protocole

L'ordre des opérations, les paramètres



Le résultat publié

Quatre mots qu'on confond souvent

Leur sens varie selon les disciplines.

En 2020, l'ACM a même **échangé** « reproduire » et « répliquer ».

	Répéter	Reproduire	Répliquer	Réutiliser
L'équipe	la même	une autre	une autre	une autre
Le dispositif	le même	le même	un autre	un autre
La question	la même	la même	la même	une nouvelle

+ de 70 %

ont échoué à reproduire l'expérience d'un collègue. Plus de la moitié, la leur.

Enquête Nature, 1 576 chercheurs, 2016 [Bak16]

32 %

des articles d'informatique dont le code a pu être récupéré et compilé en moins de 30 min

Collberg et Proebsting, 402 articles, 2016 [CP16]

Pas de fraude derrière ces chiffres : des thèses qui se terminent,
des serveurs réinstallés, des bibliothèques qui changent.

« Ça marche sur ma machine »

La phrase la plus coûteuse de la recherche calculatoire.

- 🖥️ L'expérience ne vit que sur un seul ordinateur
- 🔄 Le système et les bibliothèques changent tout seuls
- 🌙 Des dépendances installées à la main un soir, puis oubliées
- 📝 Des étapes manuelles que personne n'a écrites nulle part
- 📁 Les données ont bougé sans changer de nom



Photo NASA, IBM 704, 1957

Le premier pénalisé n'est pas votre relecteur : c'est vous, dans six mois.

2



PARTIE 2 SUR 4

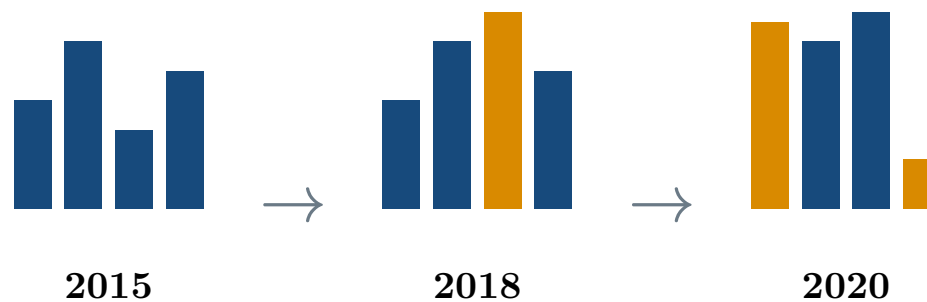
Mon terrain

Des connaissances qui changent avec la ville



Ma thèse en une diapositive

Laboratoire d'Informatique de Grenoble ■



En orange, ce qui a changé depuis la date précédente

La question

Une ville change. À quoi ressemblait ce quartier en 2015 ? Qu'est-ce qui a changé depuis ?

Ce que ça demande

Garder **toutes** les versions, et pouvoir les interroger. C'est **ConVer-G**, ma suite logicielle [Pug+26].

Des expériences fragiles



🕒 Je mesure des **durées** : très sensibles à l'environnement.

🔄 Des heures par expérience : une erreur à la fin, tout est à refaire.

Mon vrai besoin

Relancer **toute l'expérience**, sans rien réinstaller ni refaire à la main.

3



PARTIE 3 SUR 4

Ce que j'ai mis en place

Quatre couches, de la machine à l'archive



Vue d'ensemble : quatre couches

1

L'environnement Docker

Tout ce qu'il faut pour faire tourner le code tient dans un fichier texte.



2

L'exécution Kubernetes

Le même environnement tourne sur mon portable et sur le cluster, sans rien réinstaller.



3

Le protocole Argo Workflows et Hera

Le plan d'expérience devient un programme, qu'on relance en une commande.



4

La mémoire Zenodo, DOI

Le tout devient citable, daté, et ne dépend plus de mon compte personnel.



L'idée

Livrer la cuisine avec la recette, plutôt qu'écrire
« installez Java 21 et PostgreSQL 17 ».

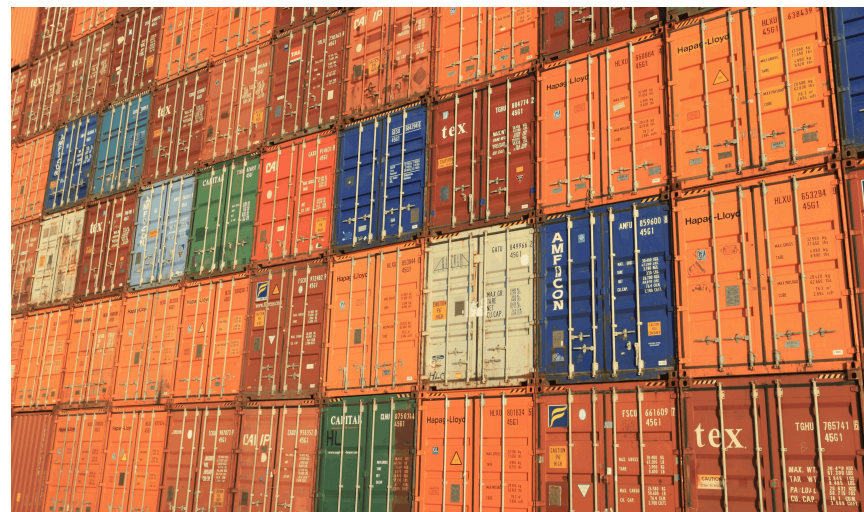
Deux mots à retenir

 Image

le plan, figé

 Conteneur

le bâtiment construit
d'après le plan, jetable
sans abîmer le plan



Un format standard, quel que soit le contenu.

Couche 1 : épingler les versions

⊗ Ce que j'écrivais au début

```
1 FROM python:latest
2 RUN pip install pandas numpy
```

latest bouge et rien n'est fixé : reconstruite en 2026, ce n'est plus la même image.

☑ Ce que j'écris maintenant

```
1 FROM python:3.11-slim@sha256:1e3f9b...
2 COPY requirements.txt .
3 RUN pip install --require-hashes \
4     -r requirements.txt
```

Même base, mêmes dépendances. L'image obtenue a une empreinte citable.

Deux règles qui m'ont évité beaucoup de nuits blanches

- 1 Jamais de données dans l'image
- 2 L'image est construite par un serveur

Couche 2 : du portable au cluster

Pourquoi j'y suis venue

Des expériences sur mon portable :

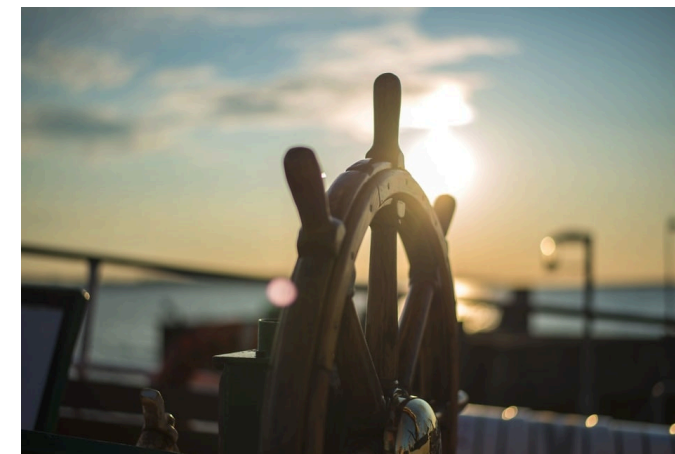
trop longues

trop gourmandes

difficiles à paralléliser

Ce que ça apporte

- 1 Redémarrage automatique en cas d'échec
- 2 Passage à l'échelle (parallélisation, configuration)

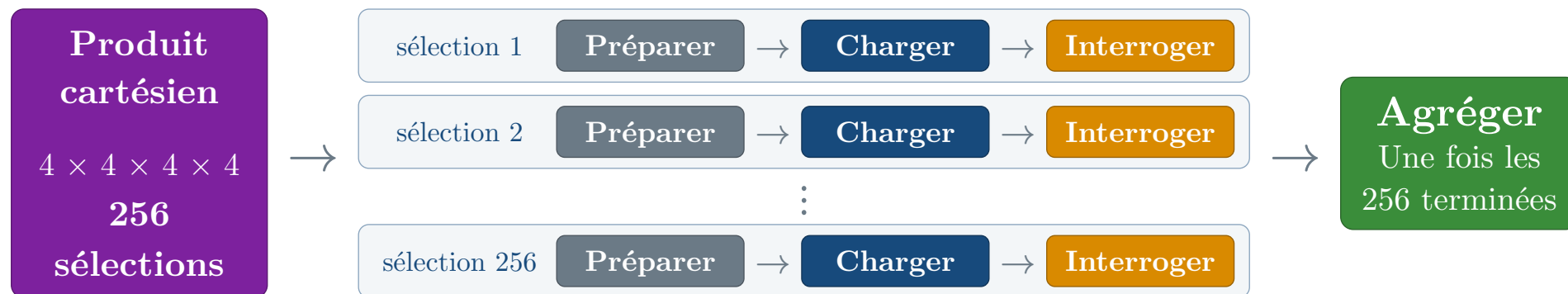


Kubernetes veut dire « pilote » en grec.

⚠ Un avertissement honnête

Le coût d'entrée est réel. Une heure sur un portable ? Restez à la couche 1.

Couche 3 : le protocole en programme



Un seul workflow paramétré, instancié 256 fois en parallèle. L'agrégation attend la fin de toutes les branches.

Configurable, donc réutilisable

Une autre campagne = une autre liste de paramètres. Le workflow ne change pas.

Couche 3 : Python plutôt que YAML

Le fichier de configuration

```
1 kind: Workflow
2 spec:
3   entrypoint: diamond
4   templates:
5     - name: diamond
6       dag:
7         tasks:
8           - name: A
9             template: echo
10          - name: B
11            dependencies: [A]
12            template: echo
13          ...
```

Chaque dépendance s'écrit à la main, ligne par ligne.

Le même plan, en Python

```
1 with Workflow(entrypoint="diamond") as w:
2     with DAG(name="diamond"):
3         A = echo(name="A")
4         B = echo(name="B")
5         C = echo(name="C")
6         D = echo(name="D")
7         A >> [B, C] >> D
8 w.create()
```

Tout le graphe tient sur une ligne, et la boucle se factorise.

Couche 4 : citable et durable

Un dépôt de code

Renommable, effaçable, sans date certaine ni identifiant stable.

Ce n'est pas une archive.

Une archive publique

Version figée + DOI [Pug26b].

Fichier de citation [Dru+21].

Une licence : sans elle, pas de réutilisation.



Software Heritage [DZ17] : l'archive universelle du code source public.



Gravure, bibliothèque de Leyde, 1610

4



PARTIE 4 SUR 4

Ce que ça coûte,
ce que ça rapporte



Les pièges où je suis tombée

Côté technique



Écrire latest : l'image change en silence



Oublier de fixer le tirage aléatoire

Côté méthode



Avoir documenté pour un relecteur imaginaire



Ne pas avoir versionné les données de test

À noter

Une durée n'est jamais reproductible au millième. Ce qui doit l'être : la **procédure** et la **conclusion**.

Par où commencer ?

Une journée

l'essentiel du bénéfice

- 1 Tester l'installation sur une machine vierge
- 2 Fixer les versions des dépendances
- 3 Ajouter une licence
- 4 Un script qui régénère toutes les figures

Une semaine

l'environnement

- 1 Mettre l'environnement dans un conteneur [KSB17]
- 2 Stocker et figer les données
- 3 Relancer le script à chaque modification

Un mois et plus

le passage à l'échelle

- 1 Un moteur de workflow : Snakemake [Möl+21], Nextflow [Di +17], Argo
- 2 Exécuter sur un cluster
- 3 Archiver, obtenir un DOI, publier le logiciel

Ce que ça a changé au quotidien

AVANT



Une semaine de bricolage
à chaque demande de révision



APRÈS



Une commande
et quelques heures d'attente



Ce que ça a coûté

≈ **2 mois**

de travail, étalés sur la thèse

- Un outillage à entretenir
- La difficulté d'expliquer pourquoi je « perdais » du temps là-dessus



Ce que ça a rapporté

- Un démonstrateur maintenu deux ans
- Une publication logicielle et des articles citables
- Des chiffres régénérables, donc des chiffres discutables

Merci, et à vous

Laboratoire d'Informatique de Grenoble ■

Pour aller plus loin

- 1 L'atelier que j'ai animé au LIRIS le 9 juin 2026
- 2 Le dépôt : un cas d'étude sur des données sismiques publiques



Dépôt de l'atelier Hera

- Ass20** Association for Computing Machinery: « Artifact Review and Badging, Version 1.1 » (2020)
- Coh+17** Cohen-Boulakia, S., Belhajjame, K., Collin, O., **et al.**: « Scientific workflows for computational reproducibility in the life sciences: Status, challenges and opportunities » **Future Generation Computer Systems**, 2017, **75**, p. 284-298.
- Mic26** Michel, F.: « Open Science, reproducible research, and the citation of articles, code and data alike », <https://doi.org/10.5281/zenodo.19557161>
- Bak16** Baker, M.: « 1,500 scientists lift the lid on reproducibility » **Nature**, 2016, **533**, (7604), p. 452-454.
- CP16** Collberg, C., Proebsting, T.A.: « Repeatability in computer systems research » **Communications of the ACM**, 2016, **59**, (3), p. 62-69.
- Pug+26** Puget Gil, J., Coquery, E., Samuel, J., Gesquière, G.: « ConVer-G: A Suite for Versioning, Querying and Visualization of Dynamic Knowledge Graphs » **Journal of Open Source Software**, 2026, **11**, (124), p. 10828.
- Boe15** Boettiger, C.: « An introduction to Docker for reproducible research » **ACM SIGOPS Operating Systems Review**, 2015, **49**, (1), p. 71-79.
- FT22** Facq, L., Théron, R.: « Introduction et bonnes pratiques de la conteneurisation » (2022)
- BP19** Bowen, A., Petazzoni, J.: « Getting started with Kubernetes and container orchestration » (2019)
- The26** The Kubernetes Authors: « Kubernetes Documentation » (2026a)
- The26** The Argo Project: « Argo Workflows: the workflow engine for Kubernetes » (2026b)
- arg26** argoproj-labs: « Hera: Python SDK for Argo Workflows » (2026)
- Pug26** Puget Gil, J.: « Atelier : construction programmatique de workflows Argo à l'aide de la librairie Hera » (2026a)
- Pug26** Puget Gil, J.: « ConVer-G (archive pérenne) », <https://doi.org/10.5281/zenodo.21724811>
- Dru+21** Druskat, S., Spaaks, J.H., Chue Hong, N., others: « Citation File Format » (2021)
- DZ17** Di Cosmo, R., Zacchiroli, S.: « Software heritage: Why and how to preserve software source code », in « iPRES 2017-14th International Conference on Digital Preservation » (2017), p. 1-10
- KSB17** Kurtzer, G.M., Sochat, V., Bauer, M.W.: « Singularity: Scientific containers for mobility of compute » **PLoS ONE**, 2017, **12**, (5), p. e0177459.

Références

- Möl+21 Mölder, F., Jablonski, K.P., Letcher, B., **et al.**: « Sustainable data analysis with Snakemake » **F1000Research**, 2021, **10**, p. 33.
- Di +17 Di Tommaso, P., Chatzou, M., Floden, E.W., Barja, P.P., Palumbo, E., Notredame, C.: « Nextflow enables reproducible computational workflows » **Nature Biotechnology**, 2017, **35**, (4), p. 316-319.